Seyed Kian Hakim (SXH200056)
CS 4395
April 12, 2025
Assignment 2 Report
GitHub URL: https://github.com/KianHm/CS-4395-A2

1: Introduction and Data

- In this project, we developed neural network architectures, such as **Feedforward Neural Network known** as **FFNN**, and **Recurrent Neural Network**, known as **RNN**. Where these networks perform on a 5-class sentiment analysis based on Yelp reviews. The main task is a standard text classification where each review must be classified by both of the networks into five sentiments. The main goal is to predict the rating $y \in \{0, 1, 2, 3, 4\}$ which is the correspondence to 1 - 5 star ratings.

- The dataset we used of Yelp reviews are split into three sections of training, testing, and validation. The **FFNN** model operates and optimizes on a bag-of-words representation, while the **RNN** model pre-trained word embedding and processes in a sequential order.

2: Implementation

- **2.1 FFNN:**

The FFNN consists of:

- An input layer corresponding to the vocabulary size
- A hidden layer with ReLU activation
- An output layer that maps to the 5 sentiment classes
- A log-softmax activation to compute class probabilities

Libraries and execution:

- We used **torch** for model construction, **torch.nn** for training, and **torch.optim** for optimization
- **Json, math, and random,** for data processing and scaling

```
def compute_Loss(self, predicted_vector, gold_label):
        return self.loss(predicted_vector, gold_label)

    def forward(self, input_vector):
        # [to fill] obtain first hidden layer representation
        h = self.activation(self.W1(input_vector))

        # [to fill] obtain output layer representation
        z = self.W2(h)

        # [to fill] obtain probability dist.
        predicted_vector = self.softmax(z)

        return predicted_vector
```

- Loss function is calculated by using the LogSoftMax function.
- Accuracy is computed per epoch for training and validation sets.
- Data processing is tokenized into a fixed-length bag of words using **convert_to_vector_representation()**.

- **2.2 RNN:**

- We implemented a **Recurrent Neural Network (RNN)** for the same 5-class sentiment classification task, using a sequential model that processes each word in a review step-by-step via a hidden state. Unlike the FFNN, which uses a bag-of-words input, the RNN model operates on **pre-trained word embeddings** loaded from **word_embedding.pkl.**

Input Format:

- Each review is tokenized and punctuation is removed.
- Each word is converted into a vector using the embedding dictionary.
- The input becomes a tensor of shape **(sequence length, 1, embedding size)**.

Libraries and execution:

- We used **torch** for model construction, **torch.nn** for training, and **torch.optim** for optimization
- **Pickle** to load the pre-trained embeddings
- **string, json, random, and argparse** for preprocessing and CLI handling

```
 def forward(self, inputs):
        # [to fill] obtain hidden layer representation
(https://pytorch.org/docs/stable/generated/torch.nn.RNN.html)
        output, hidden = self.rnn(inputs)
        # [to fill] obtain output layer representations
        z = self.W(output.squeeze(1))
        # [to fill] sum over output
        z_sum = torch.sum(z, dim=0)
        # [to fill] obtain probability dist.
        predicted_vector = self.softmax(z_sum.view(1, -1)) # To
shape (1, 5)

        return predicted_vector
```

- **RNN Layer**: Processes the input sequence step by step.
- **Linear Layer**: Projects each hidden state to a 5-class output.
- **Sum Pooling**: The output logits from each time step are summed.
- **LogSoftmax**: Produces class log-probabilities.
- **Loss Function**: nn.NLLLoss() (same as FFNN).

3: Experiment and Results

- **Evaluations:**
  To evaluate our models and analyze its performance, we used different sets of classification accuracy for both models, alongside measuring the percentage made by each model over the total number of examples in our dataset. This metric was computed on both the training and validation sets, computed after each epoch, in order to optimize

the learning progression. We tried different epoch values and hidden values for each model, in order to compare both models and also see what kind of behavior they have based on different sets.

- **Results:**

    - **FFNN (Hidden Dim = 64, Epochs = 10)**

Epoch	Training Accuracy	Validation Accuracy
1	41.68%	49.00%
2	51.03%	50.13%
3	54.14%	56.13%
4	58.76%	49.38%
5	60.63%	50.00%
6	63.14%	58.00%
7	65.52%	61.25%
8	68.26%	51.75%
9	70.29%	54.13%
10	74.10%	54.50%

    - **FFNN (Hidden Dim = 128, Epochs = 10)**

Epoch	Training Accuracy	Validation Accuracy
1	41.78%	54.25%
2	50.51%	50.38%
3	54.58%	56.38%
4	57.66%	51.38%
5	60.25%	57.13%

6	**63.70%**	**50.25%**
7	**66.34%**	**55.88%**
8	**69.28%**	**43.63%**
9	**72.92%**	**58.00%**
10	**74.28%**	**49.38%**

- **FFNN (Hidden Dim = 128, Epochs = 15)**

Epochs	Training Accuracy	Validation Accuracy
1	**41.87%**	**54.25%**
2	**50.51%**	**50.38%**
3	**54.58%**	**56.38%**
4	**57.66%**	**51.38%**
5	**60.25%**	**57.13%**
6	**63.70%**	**50.25%**
7	**66.34%**	**55.88%**
8	**69.28%**	**43.63%**
9	**72.92%**	**58.00%**
10	**74.28%**	**49.38%**
11	**77.92%**	**55.38%**
12	**81.03%**	**55.00%**
13	**83.06%**	**54.00%**
14	**85.52%**	**55.38%**
15	**86.24%**	**54.00%**

- **FFNN Insights:**

Training Accuracy improves steadily with the increase of epochs and hidden size where the validation accuracy peaks early around 6 - 7 epochs, then drops, meaning that it could overfit if the training is pushed too far.

- **RNN (Hidden Dim = 64, Epochs = 10)**

Epochs	Training Accuracy	Validation Accuracy
1	30.66%	30.13%
2	35.88%	41.38%
3	37.13%	43.75%
4	38.66%	39.25%
5	Training done to avoid overfitting!	N/A

- **RNN (Hidden Dim = 64, Epochs = 10)**

Epochs	Training Accuracy	Validation Accuracy
1	31.34%	33.88%
2	35.57%	47.63%
3	31.45%	30.13%
4	30.88%	41.50%
5	32.93%	41.88%
6	32.79%	31.00%
7	31.88%	36.88%
8	32.50%	36.38%
9	Training done to avoid overfitting!	N/A

- **RNN Insights:**
  Overall, the RNN model struggled to learn effectively, leading to early overfitting and termination. Our target was to test 10 epochs, however for different hidden dim values, it led to overfitting.

    - **FFNN VS. RNN**

Aspect	**FFNN**	**RNN**
Best Validation Accuracy	**61.25%**	**43.75%**
Overfitting Behavior	**Starts approximately at epoch 8**	**Start early at epoch 4**
Training Stability	**High**	**Low**
Data Efficiency	**High based on the results**	**Lower compared to FFNN**
Overall performance	**Best for this task**	**Struggled based on the results**

Takeaways:

- Based on our environment and tasks, the FFNN model performs better and in a more efficient way. We see in results that it has a significant advantage over the RNN model in terms of both accuracy and stability and validation.

5: Conclusion and Others

- **Conclusion:**
  By doing this assignment, we learned the characteristics of FFNN and RNN models. By implementing, testing, and analyzing, we compared both models on different sets and environments. We found that the FFNN model outperformed the RNN model in terms of validation and testing accuracy. Despitethat the RNN being better for sequential data, our results showed different outputs, where the FFNN model was more effective on our Yelp dataset.

- **Contribution and Feedback:**
  This assignment was fully done by me. It took me about 8 to 10 hours to complete it. The contribution includes, implementation, reading the code and understanding it, debugging, and most time was dedicated to testing and gathering results. The implementation was straightforward, but tuning hyperparameters and writing the results took more effort. The instructions were clear, however, I would preferred to have examples of different outputs from previous experiments, so I could have insights and understanding of the overall assignment.